

TRƯỜNG ĐẠI HỌC BÁCH KHOA - ĐẠI HỌC QUỐC GIA TP.HCM



BÀI THỰC HÀNH VI XỬ LÝ - VI ĐIỀU KHIỂN

LAB 2

NGUYỄN ĐỖ KHÁNH TRÌNH - 2353237

03/10/2025

Mục lục

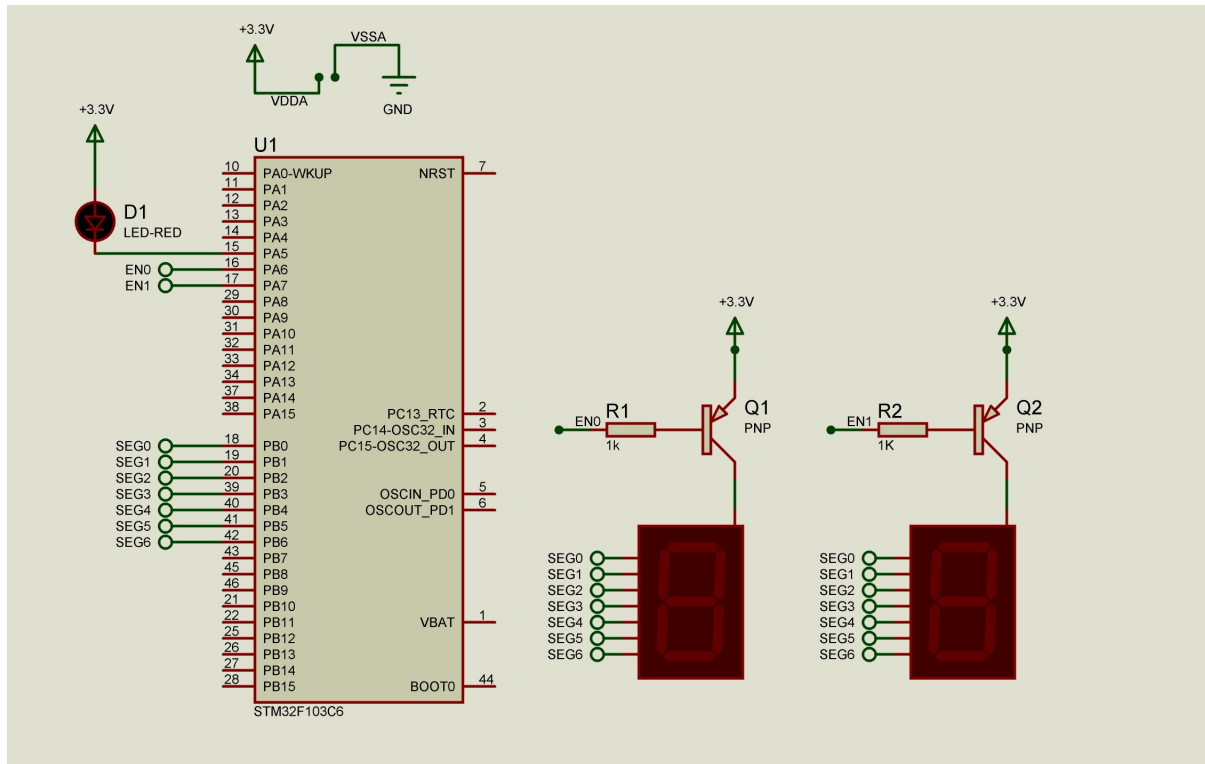
Nội Dung Thực Hành	3
1 Exercise 1	3
1.1 Report 1 - Schematic 1:	3
1.2 Report 2:	3
2 Exercise 2	5
2.1 Report 1: Schematic form exercise 2 - 8:	5
2.2 Report 2	5
3 Exercise 3	7
3.1 Report 1:	7
3.2 Report 2:	8
4 Exercise 4	8
4.1 Report 1:	8
5 Exercise 5	9
5.1 Report 1:	9
6 Exercise 6	9
6.1 Report 1:	9
6.2 Report 2:	10
6.3 Report 3:	10
7 Exercise 7-8	10
7.1 Report 1:	10
8 Exercise 9	11
8.1 Report 1: Schematic form exercise 9 - 10:	11
8.2 Report 2:	11
8.3 Exercise 10	13
8.3.1 Report 1:	13

Nội Dung Thực Hành

Code và Schematic có thể được xem trực tiếp và tải về tại GitHub Repository: Micro-controller Reports

1 Exercise 1

1.1 Report 1 - Schematic 1:



1.2 Report 2:

```
/* USER CODE BEGIN 4 */
/* Timer interrupt callback (fires every 10 ms) */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        /* existing LED_RED 1s toggle */
        counter--;
        if (counter <= 0)
        {
            counter = 100; // 100 * 10ms = 1s
            HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        }
    }
}
```

```
/* multiplex switching: toggle which display is active every 0.5s */
mux_counter--;
if (mux_counter <= 0)
{
    mux_counter = 50; // 50 * 10ms = 500 ms
    /* flip display index */
    current_display ^= 1;
    /* update displayed digit accordingly */
    show_digit_pattern(current_display);
}
}
/* USER CODE END 4 */
```

Short question: What is the frequency of the scanning process? The frequency of the scanning process can be calculated as follows:

We have the same interval $T_s = 500$ ms for each 7-segment LED to turn on before switching to the next one.

Since there are 2 LEDs used in this exercise, the total duration is

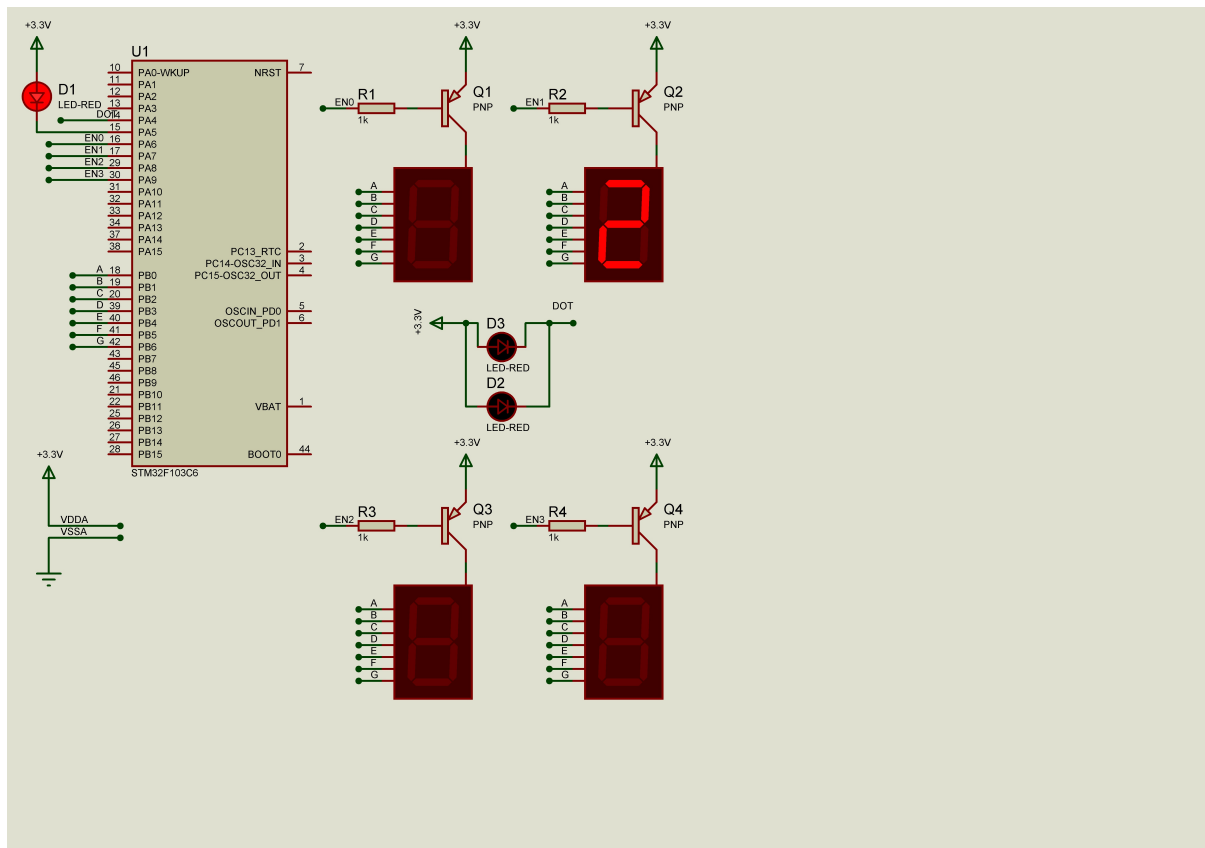
$$T_{\text{total}} = 2 \times T_s = 1 \text{ s.}$$

Therefore, the frequency of the scanning process is

$$f = \frac{1}{2T_s} = 1 \text{ Hz.}$$

2 Exercise 2

2.1 Report 1: Schematic form exercise 2 - 8:



2.2 Report 2

```

/* USER CODE BEGIN 0 */
uint8_t seg_digits[10] = {
    0b11000000, //0
    0b11111001, //1
    0b10100100, //2
    0b10110000, //3
    0b10011001, //4
    0b10010010, //5
    0b10000010, //6
    0b11111000, //7
    0b10000000, //8
    0b10010000 //9
};

void display7SEG(int firstPin, int num){
    if ( num > 9 ) return;
    uint8_t pattern = seg_digits[num];

    for ( int i = 0; i < 7; i++){

```

```
    HAL_GPIO_WritePin(GPIOB, firstPin << i, (pattern & (1 << i)) ? GPIO_PIN_SET : GPIO_PIN_RESET);
}
}
/* USER CODE END 0 */

/* USER CODE BEGIN 4 */
int currentDisplay = 3;
int counter = 0;
int digitCounter = 0;
int dotCounter = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if (htim->Instance == TIM2){

        counter++;
        if (counter >= 100){
            counter = 0;
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        }

        dotCounter++;
        if (dotCounter >= 100){
            dotCounter = 0;
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        digitCounter++;
        if (digitCounter >= 50){
            digitCounter = 0;

            switch (currentDisplay){
            case 0:
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);

                display7SEG(GPIO_PIN_0, 1);
                currentDisplay = 1;
                break;
            case 1:
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);

                display7SEG(GPIO_PIN_0, 2);
                currentDisplay = 2;
```

```

        break;
    case 2:
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);

        display7SEG(GPIO_PIN_0, 3);
        currentDisplay = 3;
        break;
    case 3:
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);

        display7SEG(GPIO_PIN_0, 0);
        currentDisplay = 0;
        break;
    }
}
}
}
/* USER CODE END 4 */

```

Short question: What is the frequency of the scanning process?

The frequency of the scanning process will be:

$$f = \frac{1}{T} = \frac{1}{0.5(s) + 0.5 \times 3(s)} = 0.5(\text{Hz})$$

3 Exercise 3

3.1 Report 1:

```

void update7SEG(int index){

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);

    switch (index){
    case 0:
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
        display7SEG(GPIO_PIN_0, led_buffer[0]);
        break;
    }
}

```

```

        case 1:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
            display7SEG(GPIO_PIN_0, led_buffer[1]);
            break;
        case 2:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
            display7SEG(GPIO_PIN_0, led_buffer[2]);
            break;
        case 3:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
            display7SEG(GPIO_PIN_0, led_buffer[3]);
            break;
        default:
            break;
    }
}

/* USER CODE BEGIN 4 */

```

3.2 Report 2:

```

int counter_led = 50;
int counter_dot = 100;
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if (htim->Instance == TIM2){
        counter_led--;
        counter_dot--;
        if (counter_led <= 0){
            counter_led = 50;
            update7SEG(index_led);
            index_led++;
            if (index_led >= MAX_LED) index_led = 0;
        }

        if (counter_dot <= 0){
            counter_dot = 100;
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        }
    }
}

/* USER CODE END 4 */

```

4 Exercise 4

4.1 Report 1:

```
/* USER CODE BEGIN 4 */
int counter = 100;
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if (htim->Instance == TIM2){
        counter--;
        if (counter <= 0){
            counter = 100;
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
            update7SEG(index_led);
            index_led++;
            if (index_led >= MAX_LED) index_led = 0;
        }
    }
}
/* USER CODE END 4 */
```

5 Exercise 5

5.1 Report 1:

```
void updateClockBuffer(int hour, int minute){
    led_buffer[0] = hour/10;
    led_buffer[1] = hour%10;
    led_buffer[2] = minute/10;
    led_buffer[3] = minute%10;
}
```

6 Exercise 6

6.1 Report 1:

If line 1 of the code above is missed, what happens after that and why?

If the line ***setTimer0(1000)*** is missed, the program, which runs under the condition that the timer's flag is set, won't execute since *timer0_counter* remains 0 (because there is no duration assigned to the function) and *timer_run()* never executes to set the *timer_flag* to be **TRUE**.

Therefore, *timer_flag* remains **FALSE**, and the LED won't blink.

6.2 Report 2:

If line 1 of the code is changed to `setTimer0(1)`, what happens after that and why?

If the code in line 1 is changed to ***setTimer0(1)***, the program will occur similarly to when that line 1 is missed (won't execute).

However, in this case, the *timer0_counter* remains 0 due to the equation integer *timer0_counter* = round down $\frac{\text{duration}}{\text{TIMER_CYCLE}} = \text{round down } \frac{1}{10} = 0$.

Therefore, *timer_flag* remains **FALSE**, and the LED won't blink either.

6.3 Report 3:

If line 1 of the code above is changed to `setTimer0(10)`, what is changed compared to the first two questions, and why?

In this case, as the *timer0_counter* = $\frac{10}{10} = 1 > 0$, it reaches the minimum duration to trigger the *timer_run()* to execute and set *timer_flag* to be **TRUE**.

So as a result, the flag is set to **TRUE**, and the LED will blink every two seconds.

7 Exercise 7-8

7.1 Report 1:

```
/* USER CODE BEGIN WHILE */
setTimer0(1000);
int hour = 15, minute = 8, second = 50;
while (1)
{
/* USER CODE END WHILE */
    if (timer0_flag == 1){
        second++;
        if (second >= 60){
            second = 0;
            minute++;
        }
        if (minute >= 60){
            minute = 0;
            hour++;
        }
        if (hour >= 24){
            hour = 0;
        }

        updateClockBuffer(hour, minute);
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
}
```

```

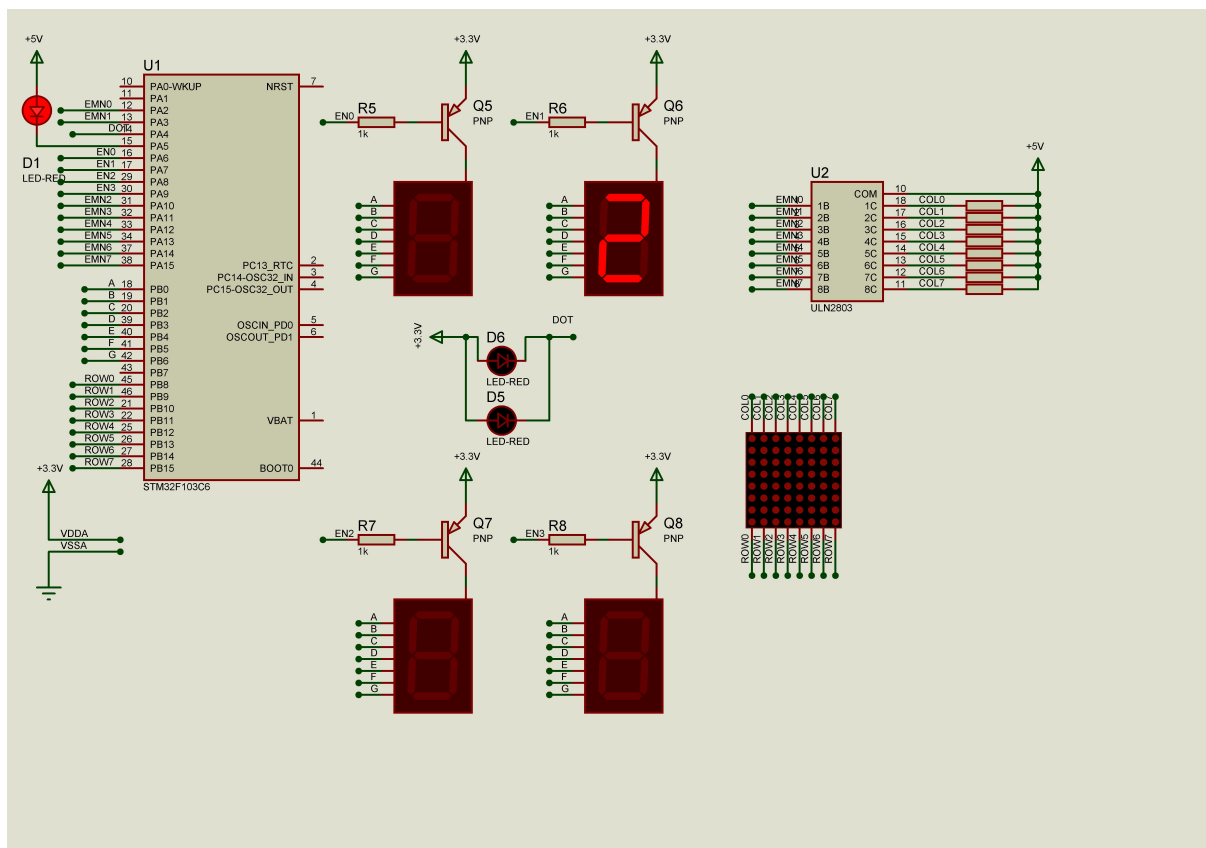
update7SEG(index_led);
index_led++;
if (index_led >= MAX_LED) index_led = 0;

setTimer0(1000);
    }
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */

```

8 Exercise 9

8.1 Report 1: Schematic form exercise 9 - 10:



8.2 Report 2:

```

const int MAX_LED_MATRIX = 8;
int index_led_matrix = 0;
uint8_t matrix_buffer[8] = {
    0b11111000,
    0b11111100,
    0b00110110,

```

```
    0b00110011 ,
    0b00110011 ,
    0b00110110 ,
    0b11111100 ,
    0b11111000
};

void updateLEDMatrix (int firstPin , int index){
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_10|
GPIO_PIN_11|GPIO_PIN_12|GPIO_PIN_13|
GPIO_PIN_14|GPIO_PIN_15, GPIO_PIN_SET);

uint8_t pattern = matrix_buffer[index];

for(int i = 0; i < 8; i++){
HAL_GPIO_WritePin(GPIOB, firstPin << i ,
(pattern & (1 << i)) ? GPIO_PIN_RESET : GPIO_PIN_SET);}

switch(index){
case 0:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_RESET);
    break;
case 1:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET);
    break;
case 2:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET);
    break;
case 3:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET);
    break;
case 4:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET);
    break;
case 5:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET);
    break;
case 6:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET);
    break;
case 7:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_RESET);
    break;
    default :
        break;
}
}
```

8.3 Exercise 10

8.3.1 Report 1:

Shift-to-right animation

```
uint8_t temp_buffer[4];

void shiftLEDMatrixRight(){

    uint8_t last = matrix_buffer[7];

    for (int i = 7; i > 0; i--) {
        matrix_buffer[i] = matrix_buffer[i - 1];
    }
    matrix_buffer[0] = temp_buffer[3];

    for (int i = 3; i > 0; i--) {
        temp_buffer[i] = temp_buffer[i - 1];
    }
    temp_buffer[0] = last;
}
```

By involving an additional temporary buffer, the character displayed on the 8x8 LED matrix can be shifted to the right by assigning the row pattern of the last column of the *led_buffer* to the first column of the *temp_buffer*, and assigning that of the first column of the *temp_buffer* to the last column of the *led_buffer*, while performing shifting other columns of both buffers to the right, which means assigning the row pattern of a column by that of its previous column.

Shift-to-left animation

```
void shiftLEDMatrixLeft(){

    uint8_t first = matrix_buffer[0];

    for (int i = 0; i < 7; i++){
        matrix_buffer[i] = matrix_buffer[i + 1];
    }

    matrix_buffer[7] = temp_buffer[0];

    for (int i = 0; i < 3; i++){
        temp_buffer[i] = temp_buffer[i + 1];
    }
    temp_buffer[3] = first;
}
```

The same method of the shift-to-right is used, but the direction and the sequence of shifting are reversed. *temp_buffer[last] := led_buffer[first]*; *led_buffer[last] :=*

temp_buffer[first]; while shifting other columns of both buffers to the left by assigning the row pattern of a column by that of its next column.